

GH raisoni COLLEGE | Engineering and Management Pune

An Empowered Autonomous Institute, Affiliated to SPPU, Pune
NAAC Accredited "A+" Grade & NBA

Department of CSE Artificial Intelligence & Machine Learning

LAB MANUAL

Subject: Database Management Systems (23UAMPCP3507)

LIST OF EXPERIMENTS

Sr. No.	Title of Experiment	CO
1	Draw E-R diagram and convert entities and relationships to relation table for a given scenario. (eg. bank, college, Employee, Hotel etc). **Virtual Lab:** https://se-iitkgp.vlabs.ac.in/exp/e-r-modeling/procedure.html.	CO1
2	To perform following SQL activity: a) Creating a database b) Creating Tables (With and Without Constraints) c) Inserting Record in table Virtual Lab**:** https://vsit.edu.in/vlab/DBMS/Views_Simulator.html.	CO2
3	To Perform the following: a. Viewing all databases, Viewing all Tables in a Database, Updating/Deleting Records in a Table.	CO2
4	To Perform the following SQL query on database: a. Altering a Table, Dropping/Truncating/Renaming Tables, Backing up / Restoring a Database.	CO2
5	For a given set of relation schemes, create tables and perform the following Simple Queries: Simple Queries with Aggregate functions, Queries with Aggregate functions (group by and having clause), Queries involving- Date Functions, String Functions, Math Functions.	CO2
6	To perform SQL query that demonstrate Join Queries- Inner Join, Outer Join, Left join, Right Join, TCL Commands.	CO2, CO3
7	To perform SQL query that demonstrate following: Search conditions, Summary queries, Sub- queries, Subqueries- With IN clause, and With EXISTS clause.	CO2
8	Write and execute PL/SQL stored procedure and function to perform a suitable task on the database. Demonstrate its use.	CO2
9	Write and execute suitable database triggers .Consider row level and statement level triggers.	CO2
10	Create MongoDB Document & Implement CURD Operations on it.	CO4
	Content Beyond Syllabus	
11	Design database using MySQL & connectivity with java/python.	CO2
12	Study of Datalib Architecture.	CO1

Experiment No.1

TITLE

Design and Conversion of ER Diagrams to Relational Tables for Two Real-World Scenarios.

PROBLEM STATEMET

Analyse two different real-world scenarios—such as a bank and a college to:

1. Design an Entity-Relationship (ER) diagram that captures entities, attributes, and their relationships.
2. Convert the ER diagram into a set of relational tables, specifying primary keys (PK), foreign keys (FK), and composite keys, according to ER-to-Relational transformation rules.

OBJECTIVE

- Identify entities, their attributes (including PKs), and types of relationships (1:1, 1:N, M:N).
- Draw ER diagrams using standard notation (e.g., Chen or Crow's Foot).
- Apply mapping rules to convert each ER diagram to relational schema.
- Define tables for entities, weak entities, associative relationships (junction tables), with correct PKs and FKs.
- Show sample schemas with field definitions and key constraints.
- Explain the conversion process and justify design decisions.

PLATFORM REQUIRED

- **Operating System:** Windows or Linux
- **Software/Tools:** Diagramming tools (e.g., MySQL Workbench, draw.io), RDBMS platforms (e.g., MySQL, PostgreSQL), optional environments like Jupyter Notebook or Lucidchart for documentation and visualization.

THEORY

Entity-Relationship (ER) Diagrams

Introduced by Peter Chen (1976), ER diagrams are visual models representing entities (objects), their attributes, and inter-entity relationships.

Entities are shown as rectangles, attributes as ovals (with PK underlined), and relationships as diamonds (Chen notation) or lines with cardinality markers (Crow's Foot).

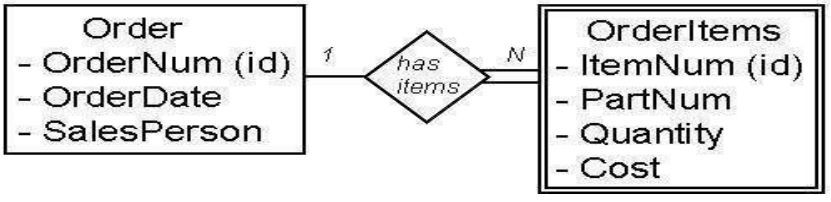
Entity Relationship Diagram Symbols & Notations mainly contains three basic symbols which are rectangle, oval and diamond to represent relationships between elements, entities and attributes. There are some sub-elements which are based on main elements in ERD Diagram. ER Diagram is a visual representation of data that describes how data is related to each other using different ERD Symbols and Notations.

Following are the main components and its symbols in ER Diagrams:

- **Rectangles:** This Entity Relationship Diagram symbol represents entity types
- **Ellipses :** Symbol represent attributes
- **Diamonds:** This symbol represents relationship types
- **Lines:** It links attributes to entity types and entity types with other relationship types
- **Primary key:** attributes are underlined
- **Double Ellipses:** Represent multi-valued attributes



Figure 1. ER Diagram Symbols

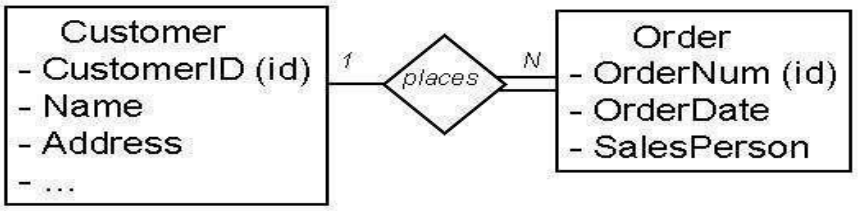


Chen Notation

a. **ORDER** (OrderNum (key), OrderDate, SalesPerson)
ORDERITEMS (OrderNum (key)(fk) , ItemNum (key), PartNum, Quantity, Cost)

b. In the above example, in the ORDERITEMS Relation: OrderNum is the ForeignKey and OrderNum plus ItemNum is the Composite Key.

Chen Notation



In the ORDER Relation: OrderNum is the *Key*.

Representing Relationships

b. **1:1** Relationships. The key of one relation is stored in the second relation. Look at example queries to determine which key is queriedmost often.

c. **1:N** Relationships.
Parent - Relation on the "1" side. **Child** - Relation on the "Many" side.

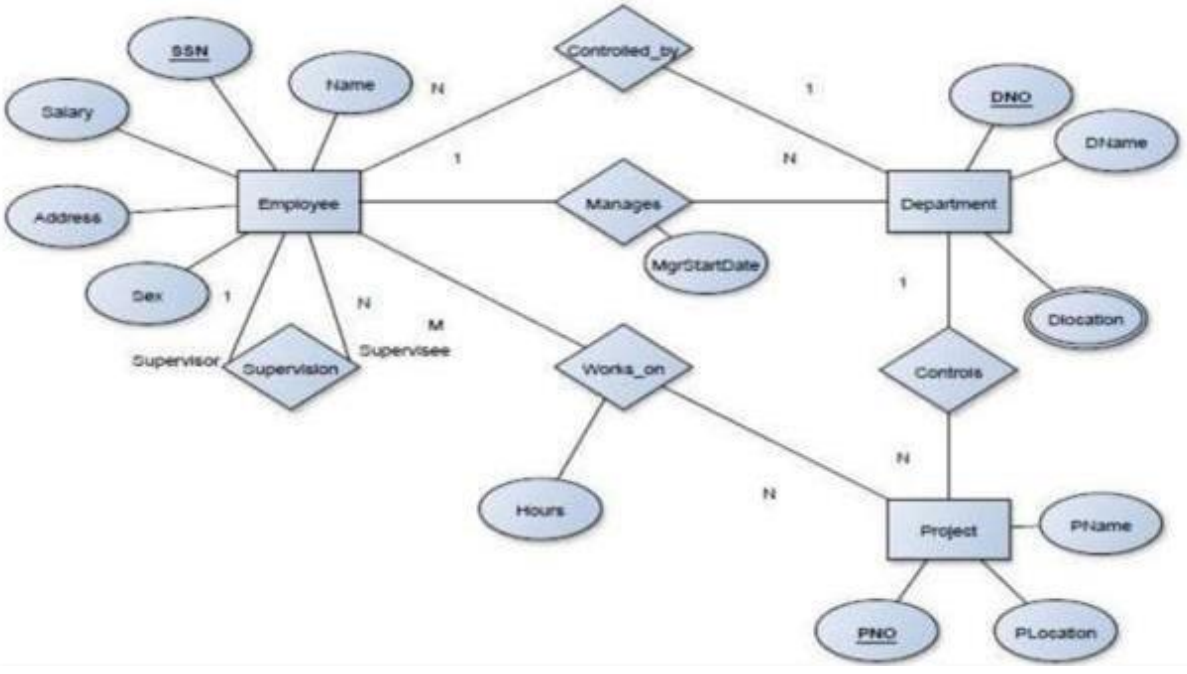
d. Represent each Entity as a relation.
Copy the key of the parent into the child relation.

e. CUSTOMER (CustomerID (key), Name, Address, ...)
ORDER (OrderNum (key), OrderDate, SalesPerson,

CustomerID (fk))Schema for Company Database:

EMPLOYEE (SSN, Name, Address, Sex, Salary, SuperSSN, DNo)DEPARTMENT (DNo, DName, MgrSSN, MgrStartDate) DLOCATION (DNo, DLoc)
PROJECT (PNo, PName, PLocation, DNo) WORKS_ON (SSN, PNo, Hours)

ER-Diagram:



STEP BY STEP ALGORITHM

- **Identify two real-world scenarios** (e.g., Bank System, College System).
- Define **entities with attributes and keys.**
- **Determine relationships and cardinalities** (1:1, 1:N, M:N).
- **Draw ER diagrams** using chosen notation.
- **Transform ER diagrams to relational tables**:
- Entity tables with PK
- Add FK according to relationships
- Create associative tables for M:N
- Convert multi-valued/composite attributes
- **Document schemas** with field lists, data types, key assignments.
- **Describe mapping rationale** and foreign key constraints.

EXAMPLE SCENARIOS & TABLES

Scenario 1: Bank Management System

Entities & Keys:

- **Bank** (BankID PK, Name, Address)
- **Branch** (BranchID PK, BankID FK → Bank, Address)
- **Customer** (CustomerID PK, Name, Phone, Address)
- **Account** (AccountNo PK, Type, Balance, BranchID FK → Branch)
- **Loan** (LoanID PK, Type, Amount, BranchID FK → Branch)

Relationships:

- Bank–Branch (1:N) → Branch.BANKID as FK
- Branch–Account (1:N) → Account.BRANCHID as FK
- Branch–Loan (1:N) → Loan.BRANCHID as FK
- Customer–Account (M:N) → junction: **CustomerAccount**(CustomerID, AccountNo)

Resulting Tables:

Bank(BankID PK, Name, Address)
Branch(BranchID PK, Address, BankID FK)
Customer(CustomerID PK, Name, Phone, Address)
Account(AccountNo PK, Type, Balance, BranchID FK)
Loan(LoanID PK, Type, Amount, BranchID FK)
CustomerAccount(CustomerID PK, AccountNo PK)

Scenario 2: College Enrollment System

Entities & Keys:

- **Student** (StudentID PK, Name, DOB, Major)
- **Course** (CourseID PK, Title, Credits)
- **Instructor** (InstructorID PK, Name, Dept)
- **Department** (DeptID PK, Name, Office)

Relationships:

- Department–Instructor (1:N) → Instructor.DEPTID FK
- Instructor–Course (M:N); handled via `Teaches` associative table
- Student–Course (M:N); handled via `Enrollment` associative table (may include Grade)

Resulting Tables:

- `Department(DeptID PK, Name, Office)`
- `Instructor(InstructorID PK, Name, DeptID FK)`
- `Course(CourseID PK, Title, Credits)`
- `Student(StudentID PK, Name, DOB, Major)`
- `Teaches(InstructorID PK, CourseID PK)`
- `Enrollment(StudentID PK, CourseID PK, Grade)`

QUESTIONS

1. What is an ER diagram, and why is it essential for database design?
2. Explain the purpose of underlining attributes in ER diagrams.
3. How do you choose between Chen and Crow's Foot notation?
4. Describe the rules for converting 1: N, 1:1, and M: N relationships.
5. How are weak entities handled during conversion?
6. Why do associative tables appear in relational schemas?
7. What is the importance of defining foreign keys in relational tables?

CONCLUSION

Experiment No.2

TITLE

Performing SQL DDL and DML with Constraints.

PROBLEM STATEMET

Execute a series of SQL tasks to manage database structure and integrity:

1. Create a new database.
2. Define tables both with and without constraints.
3. Insert records and modify table definitions using SQL DDL and DML statements.

OBJECTIVE

- Create a database using `CREATE DATABASE`.
- Define tables without constraints for baseline understanding.
- Redefine or alter tables to enforce data integrity using constraints:
  - NOT NULL
  - UNIQUE
  - DEFAULT
  - CHECK
  - PRIMARY KEY, FOREIGN KEY
- Insert data into tables and validate behaviours.
- Modify tables to add/drop columns or constraints (`ALTER TABLE`).
- Remove data and tables using `TRUNCATE` and `DROP TABLE`.

PLATFORM REQUIRED

- **Operating System:** Windows or Linux
- **Software/Tools:** MySQL, Oracle, SQL Server, or PostgreSQL; SQL client or command-line interface

THEORY

SQL Constraints

Constraints enforce rules at the schema level to preserve data validity. Common types include:

- **NOT NULL:** Ensures a column does not accept NULL values.
- **UNIQUE:** Requires unique values across a column or group of columns; allows one NULL
- **DEFAULT:** Supplies a default value if no value is provided during insertion.
- **CHECK:** Enforces a boolean expression on column values (e.g. `salary >= 0`).
- **PRIMARY KEY:** A special UNIQUE + NOT NULL constraint; identifies each row uniquely. Only one per table.
- **FOREIGN KEY:** Ensures referential integrity by linking to a primary key in another table

STEP BY STEP ALGORITHM

1. **Create a new database:**

   CREATE DATABASE a1;

   USE a1;

2. **Create tables without constraints:**

   CREATE TABLE student (rollno INT(5), name VARCHAR(30));

3. **Create tables with various constraints:**

```
CREATE TABLE emp (
  empno NUMBER(6),
  ename VARCHAR2(20) NOT NULL,
  job VARCHAR2(10) NOT NULL,
  deptno NUMBER(3),
  sal NUMBER(7,2),
  experience NUMBER(2),
  CONSTRAINT emp_pk PRIMARY KEY (empno),
  CONSTRAINT emp_ck_empno CHECK (empno > 100)
);
```

4. **Alter tables to add/drop columns or modify constraints:**

   a) Add column:

   ALTER TABLE emp ADD (experience NUMBER(2));

   b) Modify column:

   ALTER TABLE emp MODIFY (job VARCHAR2(12));

   c) Add CHECK constraint:

   ALTER TABLE emp ADD CONSTRAINT emp_ck_experience CHECK (experience >= 0);

   d) Drop column:

   ALTER TABLE emp DROP COLUMN experience;

5. **Create another table with foreign key constraint:**

```
CREATE TABLE dept (
  deptno NUMBER(2) PRIMARY KEY,
  dname VARCHAR2(10),
  loc VARCHAR2(10)
);
CREATE TABLE emp (
  empno NUMBER(5),
  ename VARCHAR2(25) NOT NULL,
  job VARCHAR2(15),
  mgr NUMBER(5),
  joindate DATE,
  salary NUMBER(7,2),
  comm NUMBER(7,2),
  deptno NUMBER(7),
  CONSTRAINT emp_fk_dept FOREIGN KEY (deptno) REFERENCES dept(deptno)
);
```

6. **Insert sample records and verify constraints:**

   INSERT INTO dept VALUES (12, 'computer', 'pune');

   INSERT INTO emp VALUES (23, 'shiv', 'manager', 55, '2019-09-12', 30000, 55, 12);

7. **Clean up data and structures:**

   TRUNCATE TABLE emp;
   DROP TABLE emp;
   DROP TABLE dept;

QUESTIONS

1. What roles do different constraints play in maintaining data integrity?
2. When would you use `NOT NULL` versus `CHECK` constraints?
3. Why is a primary key both unique and non-null by definition?
4. How does a foreign key enforce relationships between tables?
5. How does `DEFAULT` simplify data entry?
6. What happens if you try to insert data that violates a constraint?
7. Why differentiate between `TRUNCATE` and `DROP TABLE`?

CONCLUSION

Experiment No.3

TITLE

To Perform the Following Database Operations:

- Viewing All Databases
- Viewing All Tables in a Database
- Updating/Deleting Records in a Table

PROBLEM STATEMENT

The objective of this task is to explore basic database management operations using SQL. This includes viewing the list of available databases, listing all tables within a selected database, and performing operations to update and delete records in specific tables. These operations form the foundation of interacting with relational databases and managing data effectively.

OBJECTIVE

- Connect to a database management system (DBMS) such as MySQL or SQLite.
- View all databases available on the server.
- View all tables present in a selected database.
- Execute SQL queries to update specific records in a table.
- Execute SQL queries to delete specific records from a table.
- Ensure data integrity and security while performing updates or deletions.

PLATFORM REQUIRED

- **Operating System:** Windows or Linux
- **Software/Tools:** MySQL Workbench / SQLite / PostgreSQL / SQL Server / phpMyAdmin / Any DBMS Client
- **Languages:** SQL

THEORY

Database

A database is a structured collection of data that can be easily accessed, managed, and updated. Databases are managed using Database Management Systems (DBMS) such as MySQL, PostgreSQL, and SQLite.

SQL (Structured Query Language)

SQL is the standard language used for managing and manipulating relational databases.

Common SQL operations include:

- Viewing databases and tables
- Inserting data
- Updating records
- Deleting records
- Retrieving records using queries

Common SQL Commands for This Task

1. Viewing All Databases

To view all the databases available on a server:

```
SHOW DATABASES;
```

2. Viewing All Tables in a Database

To view all tables in a selected database:

```
USE database_name;
SHOW TABLES;
```

3. Viewing Records in a Table

To inspect data in a table:

```
SELECT * FROM table_name;
```

4. Updating Records in a Table

To update existing data in a table:

```
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;
```

5. Deleting Records from a Table

To delete specific records:

```
DELETE FROM table_name
WHERE condition;
```

STEP-BY-STEP ALGORITHM

1. Open the DBMS client (e.g., MySQL Workbench, SQLite Browser).
2. Connect to the database server.

3. Execute SHOW DATABASES; to list all databases.
4. Select a specific database using the USE command.
5. Run SHOW TABLES; to view all tables within the selected database.
6. Use SELECT * FROM table_name; to view data in a table.
7. Perform record updates using the UPDATE command.
8. Perform deletions using the DELETE command.
9. Use SELECT queries to verify the changes.

QUESTIONS

1. What is a Database?
2. What is SQL and why is it important in database management?
3. How do you list all databases available on a server?
4. How can you view the tables within a selected database?
5. Explain the difference between the UPDATE and DELETE SQL commands.
6. What precautions should you take before updating or deleting records in a table?
7. Why is it important to use a WHERE clause in UPDATE and DELETE statements?

CONCLUSION

Experiment No.4

TITLE

To Perform the Following Database Operations:

- Altering a Table
- Dropping, Truncating, and Renaming Tables
- Backing Up and Restoring a Database

PROBLEM STATEMENT

The objective of this task is to explore advanced database structure management operations using SQL. This includes altering existing table structures, removing or renaming tables, and performing essential backup and restore operations for database maintenance. These tasks are crucial for managing schema changes, conserving storage, and ensuring data recovery and security.

OBJECTIVE

- Connect to a database management system (DBMS) such as MySQL or PostgreSQL.
- Alter the structure of an existing table (e.g., add, modify, or delete columns).
- Drop or truncate tables as needed.
- Rename existing tables within a database.
- Perform database backup operations to preserve data.
- Restore a database from a backup file.
- Understand the implications of structural changes and ensure data integrity during these operations.

PLATFORM REQUIRED

- **Operating System:** Windows or Linux
- **Software/Tools:** MySQL Workbench / PostgreSQL / SQL Server / SQLite / phpMyAdmin
- **Languages:** SQL

THEORY

1. Altering Tables

Used to change the structure of an existing table without deleting the table data. Common operations:

```
ALTER TABLE table_name ADD column_name datatype;
ALTER TABLE table_name MODIFY column_name new_datatype;
ALTER TABLE table_name DROP COLUMN column_name;
```

2. Dropping a Table

Completely removes a table and all its data from the database:

```
DROP TABLE table_name;
```

3. Truncating a Table

Deletes all rows from a table but retains the table structure:

```
TRUNCATE TABLE table_name;
```

4. Renaming a Table

Changes the name of an existing table:

```
RENAME TABLE old_table_name TO new_table_name;
```

5. Backing Up a Database

Creates a copy of the database for safekeeping. In MySQL, for example:

```
mysqldump -u username -p database_name > backup_file.sql
```

6. Restoring a Database

Restores a database from a previously saved backup file:

```
mysql -u username -p database_name < backup_file.sql
```

STEP-BY-STEP ALGORITHM

1. Launch your DBMS tool (e.g., MySQL Workbench, phpMyAdmin).
2. Connect to the database server using proper credentials.
3. Select the working database using:
   USE database_name;
4. Perform table structure changes using ALTER TABLE statements.
5. Use DROP or TRUNCATE to delete table data or structure.
6. Rename any tables using the RENAME TABLE command.
7. Backup the database using tools like mysqldump or GUI options.
8. Restore the database using command-line tools or import functionality in GUI.
9. Verify all operations with SELECT and SHOW commands to ensure changes are applied correctly.

QUESTIONS

1. What is the purpose of the ALTER TABLE command in SQL?
2. How is TRUNCATE different from DELETE and DROP?
3. What happens when you drop a table?
4. Why might you need to rename a table in a database?
5. What are the risks associated with altering or dropping tables?
6. Explain the process of backing up a database.
7. How do you restore a database from a backup file?

CONCLUSION

Experiment No. 5

TITLE

Experiment: SQL Queries with Aggregate, Date, String, Math, and Set Operations

PROBLEM STATEMET

For a given relational schema, design and execute SQL queries that demonstrate:

1. **Aggregate functions** (e.g., `COUNT`, `SUM`, `AVG`, `MAX`, `MIN`)
2. **Grouping and filtering** with `GROUP BY` and `HAVING`
3. **Date functions** (e.g., `CURDATE()`, `DATEDIFF()`, `DAY()`, `MONTH()`)
4. **String functions** (e.g., `UPPER()`, `LOWER()`, `CONCAT()`, `SUBSTR()`)
5. **Math functions** (e.g., `ROUND()`, `FLOOR()`, `POWER()`)
6. **Set operators** (`UNION`, `INTERSECT`, `EXCEPT`)

OBJECTIVE

- Use various **SQL aggregate functions** to summarize data.
- Apply `GROUP BY` and `HAVING` to filter grouped results.
- Demonstrate **date manipulation** and extraction using built-in functions.
- Manipulate text using **string functions**.
- Perform numeric calculations using **math functions**.
- Combine multiple query results using **set operators**.
- Validate correctness by examining query outputs from a sample `product` table.

PLATFORM REQUIRED

- **Operating System:** Windows or Linux
- **Software/Tools:** MySQL, PostgreSQL, SQL Server, or Oracle; SQL client or interactive console.

THEORY

Aggregate Functions & Grouping

Aggregate functions compute a single summary value from a set of rows. Common examples include COUNT, SUM, AVG, MIN, MAX. They are often paired with GROUP BY to perform summary operations per group, and HAVING to filter based on aggregate results (e.g., departments having more than 1000 in sales)

Date Functions

Databases provide date functions to extract components (`DAY()`, `MONTH()`, `YEAR()`), compute intervals (`DATEDIFF()`), and perform arithmetic (`DATE_ADD()`, `DATE_SUB()`) .

String Functions

Common string operations include converting cases (`UPPER()`, `LOWER()`), trimming whitespace (`LTRIM()`, `RTRIM()`), concatenating (`CONCAT()`), substring extraction (`SUBSTR()`), and replacing text (`REPLACE()`)

Math Functions

Use functions like `ROUND()`, `FLOOR()`, `CEILING()`, `POWER()`, `SQRT()`, and `LOG()` for numerical calculations

Set Operators

Operations like `UNION`, `INTERSECT`, and `EXCEPT` combine or compare the results of two `SELECT` queries, provided their column types match.

STEP BY STEP ALGORITHM

1. **Create relation schemes and sample data**

```
CREATE TABLE product(id INT, pname VARCHAR(50), pprice DECIMAL(8,2), discount INT, created DATE);
INSERT INTO product VALUES
 (1,'AMIT',2000,100,'2025-07-01'),
 (2,'CORNFLAKES',800,50,'2025-06-15'),
(3,'PARLE',90,5,'2025-05-20'),
 (4,'NESTLE',70,2,'2025-04-10'),
 (5,'CADBURY',100,10,'2025-03-05');
```

2. **Execute aggregate queries**

```
SELECT COUNT(id) AS numProducts,
    SUM(pprice) AS totalPrice,
    AVG(pprice) AS avgPrice,
    MAX(discount) AS maxDisc,
    MIN(pprice) AS minPrice
FROM product;
```

3. **Group by and having**

```
SELECT discount, COUNT(*) AS cnt, AVG(pprice) AS avgPrice
FROM product
GROUP BY discount
HAVING COUNT(*) > 1;
```

4. **Date function queries**

```
SELECT DAY(created) AS day,
    MONTH(created) AS month,
    YEAR(created) AS year,
    DATEDIFF(CURDATE(), created) AS daysOld
FROM product;
```

5. **String function queries**

```
SELECT UPPER(pname), LOWER(pname),
    CONCAT(pname,'_2025') AS tag,
    SUBSTR(pname, 1, 3) AS prefix,
    REPLACE(pname,'PAR','KIT')
FROM product;
```

6. **Math function queries**

```
SELECT ROUND(pprice*0.9, 2) AS discountedPrice,
    POWER(pprice, 2) AS priceSquared,
    FLOOR(pprice/100)*100 AS floorHundred
FROM product;
```

7. **Set operator usage**

```
SELECT pname FROM product WHERE discount > 50
UNION
SELECT pname FROM product WHERE pprice < 100;
```

QUESTIONS

1. What does each aggregate function (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`) compute?
2. How do `GROUP BY` and `HAVING` improve summary-level querying?
3. Why is filtering done in `HAVING` rather than `WHERE` post-aggregation?
4. How would you calculate the age of an item in days using `DATEDIFF()`?
5. Which built-in functions convert string cases and why?
6. Explain the role of `ROUND()`, `FLOOR()`, and `POWER()` in SQL math expressions.
7. How do `UNION`, `INTERSECT`, and `EXCEPT` combine query results?
8. Explain the significance of visualizing regression results.

CONCLUSION

Experiment No. 6

TITLE

Demonstrating SQL Join Queries – Inner, Outer, Left, and Right Joins.

PROBLEM STATEMET

Perform various SQL JOIN operations (Inner Join, Non-Equi Join, Self-Join, Left Join, Right Join, Full Outer Join) to combine and analyze related data across multiple tables.

OBJECTIVE

- Understand different types of SQL joins: Inner (Equi), Non-Equi, Self, Outer (Left, Right, Full).
- Load and populate tables (`emp`, `dept`, `salgrade`, `employee`, `department`, `dlocation`, `project`, `works on`, `orders`, `customers`).
- Execute JOIN queries to:
    - Retrieve matching rows across tables.
    - Demonstrate non-equi and self-joins.
    - Show left, right, and full outer joins.
- Analyse real-world queries combining employees, departments, projects, and customer orders.
- Observe how join type affects resulting dataset—all rows vs. matched rows.

PLATFORM REQUIRED

- **Operating System:** Windows or Linux.
- **Software/Tools:** MySQL, Oracle SQL Developer, or any SQL RDBMS.

THEORY

SQL JOIN

A `JOIN` clause combines rows from two or more tables based on related columns.

Types of Joins

1. **Equi-Join / Inner Join**

   Uses the equal (=) operator to combine rows from two tables only when matching values exist.

2. **Non-Equi-Join**

   Uses relational operators (<, >, <=, >=, !=) in join conditions—for instance, matching salary ranges.

3. **Self-Join**

   A table joins with itself to compare rows—e.g., linking employees to managers within the same `emp` table.

4. **Outer Join**

   Returns matched rows plus unmatched rows from one or both tables:

   - **Left Join**: all rows from the left table + matching rows from the right.
   - **Right Join**: all rows from the right table + matching rows from the left.
   - **Full Outer Join**: combined set of left and right joins

IMPLEMENTATION & EXAMPLES

1. Equi-Join / Inner Join

```
SELECT e.empno, e.ename, e.dept, d.deptno, d.loc, d.dname
FROM emp e JOIN dept d ON e.dept = d.deptno;
```

Outputs employee–department matches.

2. Non-Equi-Join (Range Satisfying)

```
SELECT e.ename, e.salary, s.grade
FROM emp e JOIN salgrade s
ON e.salary >= s.losal AND e.salary <= s.hisal;
```

3. Self-Join (Employee–Manager Relationship)

```
SELECT worker.ename AS employee,
    manager.ename AS manager
FROM emp worker JOIN emp manager
ON worker.mgr = manager.empno;
```

4. Outer Joins

Left Join:

```
SELECT o.orderID, c.cust_name, c.city
FROM orders o LEFT JOIN customers c
ON o.custID = c.custID;
```

Right Join:

```
SELECT o.orderID, c.cust_name, c.city
FROM orders o RIGHT JOIN customers c
ON o.custID = c.custID;
```

5. Full Outer Join / Union Strategy

Combine left and right joins using UNION to simulate full outer join in systems lacking direct support.

6. Complex Real-World Queries

Project involvement by employee or manager "Scott":

```
(SELECT DISTINCT p.PNO FROM PROJECT p
  JOIN DEPARTMENT d ON p.DNO = d.DNO
  JOIN EMPLOYEE e ON d.MGRSSN = e.SSN
  WHERE e.NAME = 'SCOTT')
UNION
(SELECT DISTINCT w.PNO FROM PROJECT p
  JOIN WORKS_ON w ON p.PNO = w.PNO
  JOIN EMPLOYEE e ON w.SSN = e.SSN
  WHERE e.NAME = 'SCOTT');
```

10% Raise for employees on project 'IoT':

```
SELECT E.NAME, 1.1 * E.SALARY AS INCREASED_SALARY
FROM EMPLOYEE E
 JOIN WORKS_ON W ON E.SSN = W.SSN
JOIN PROJECT P ON W.PNO = P.PNO
WHERE P.PNAME = 'IOT';
```

Aggregate salary metrics in 'Accounts' department:

```
SELECT SUM(E.SALARY), MAX(E.SALARY), MIN(E.SALARY), AVG(E.SALARY) FROM
EMPLOYEE E JOIN DEPARTMENT D ON E.DNO = D.DNO WHERE D.DNAME = 'ACCOUNTS';
```

QUESTIONS

1. What is an SQL JOIN and its purpose?
2. Differentiate Equi-Join from Non-Equi-Join.
3. What is a Self-Join and when is it used?
4. Explain Left, Right, and Full Outer Joins.
5. In what scenarios would you use a Non-Equi-Join?
6. How do you simulate a Full Outer Join in SQL engines without direct support?
7. Why are aggregate functions useful in combination with joins?

CONCLUSION

Experiment No. 7

TITLE

To perform SQL query that demonstrate following-Search conditions, Summary queries, Sub- queries,

Subqueries- with IN clause, With EXISTS clause**.**

PROBLEM STATEMENT

Take a database with employee and department information. The task is to write SQL queries to filter employees using search conditions, calculate summaries like average salary per department, and use subqueries to extract data based on nested criteria. Specifically, you will use subqueries with IN and EXISTS clauses to find employees in certain departments or locations, demonstrating your understanding of advanced SQL querying techniques.

OBJECTIVE

- To learn how to apply search conditions to filter data using the WHERE clause.
- To use aggregate functions for summary queries such as COUNT(), AVG(), SUM(), MAX(), and MIN().
- To understand and implement subqueries for nested data retrieval.
- To demonstrate the use of subqueries with the IN clause for matching multiple values.
- To demonstrate the use of subqueries with the EXISTS clause for checking row existence.

PLATFORM REQUIRED

- **Relational Database Management System (RDBMS):**

  Oracle Database (for PL/SQL) / MySQL / PostgreSQL / SQL Server
- **SQL Client Tools:**

  Oracle SQL Developer / MySQL Workbench / pgAdmin / SQL Server Management Studio
- **Operating System:**

  Windows / Linux / macOS
- **Additional Tools:**

  Text Editor (optional for writing SQL scripts)

THEORY

1. Search Conditions

Search conditions in SQL are used to filter records based on specified criteria using the WHERE clause. Conditions can use comparison operators (=, <>, <, >, etc.), logical operators (AND, OR, NOT), and pattern matching (LIKE). These conditions help retrieve only those rows that meet certain criteria.
Example:

```
SELECT * FROM employees WHERE salary > 50000 AND department_id = 2;
```

2. Summary Queries

Summary queries aggregate data to provide insights, often grouped by specific columns using GROUP BY. Aggregate functions include:

- COUNT() — counts rows

- SUM() — totals values
- AVG() — averages values
- MAX() — finds maximum value
- MIN() — finds minimum value

Example:

SELECT department_id, AVG(salary) FROM employees GROUP BY department_id;

3. Subqueries

A subquery is a query nested inside another SQL query. It can return single or multiple values and is useful for complex filtering and data retrieval.

Example:

SELECT * FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);

4. Subqueries with IN Clause

The IN clause tests if a value matches any value in a list or the result of a subquery. It's used to filter rows based on multiple values returned by the subquery.

Example:

SELECT * FROM employees WHERE department_id IN (SELECT department_id FROM departments WHERE location = 'New York');

5. Subqueries with EXISTS Clause

The EXISTS clause checks for the existence of rows returned by a subquery. It returns TRUE if the subquery returns at least one row, making it useful for correlated subqueries where the outer query depends on the inner query.

Example:

SELECT * FROM employees e WHERE EXISTS (SELECT 1 FROM departments d WHERE d.department_id = e.department_id AND d.location = 'New York');

These SQL features are fundamental for data retrieval and manipulation in relational databases, allowing you to write powerful and efficient queries.

STEP-BY-STEP ALGORITHM:

- Start
- Connect to the Database
- Execute Search Condition Query

- Use SELECT statement with WHERE clause to filter data based on given conditions.
- Execute Summary Query

Use aggregate functions (COUNT, SUM, AVG, MAX, MIN) with GROUP BY to summarize data.

- **Execute Subquery**

- Write a SELECT query with a nested subquery to filter data based on values retrieved from the subquery.
- **Execute Subquery with IN Clause**
- Use IN operator with a subquery to select rows where a column matches any value from the subquery result.
- **Execute Subquery with EXISTS Clause**
- Use EXISTS operator with a subquery to check if at least one matching row exists in the subquery for each row of the outer query.
- **Display the Results**
- **End**

QUESTIONS

- What is the purpose of the WHERE clause in SQL? Give an example.
- How do aggregate functions like SUM() and AVG() help in summary queries?
- Write an SQL query to find employees whose salary is greater than the average salary of all employees.
- Explain the difference between subqueries using IN and EXISTS clauses. Give an example of each.
- Write a query to list all employees who work in departments located in 'New York' using a subquery with the IN clause.
- Write a query to find all employees for whom a corresponding department exists in the 'HR' department using the EXISTS clause.

CONCLUSION

Experiment No. 8

TITLE

Write and execute PL/SQL stored procedure and function to perform a suitable task on the database. Demonstrate its use.

PROBLEM STATEMENT

Create and execute a PL/SQL stored procedure to update employee salaries by a percentage and a function to calculate the average salary of a given department. Demonstrate their usage on the employee database.

OBJECTIVE

- Understand the concept of PL/SQL stored procedures and functions.
- Write and execute a stored procedure to perform bulk salary updates.
- Write and execute a function to compute average salary per department.
- Learn how to call procedures and functions in PL/SQL.
- Analyze the effect of procedures and functions on the database.

.

PLATFORM REQUIRED

- **Oracle Database** (11g, 12c, 19c or later)
- **Oracle SQL Developer** or **SQL*Plus** (for running PL/SQL code)
- Operating System: Windows / Linux / macO

THEORY

- **PL/SQL** is Oracle's procedural extension to SQL, allowing procedural constructs such as loops, conditions, and modular programming through stored procedures and functions.
- A **Stored Procedure** is a named PL/SQL block that performs actions like data manipulation and can accept parameters.
- A **Function** is similar to a procedure but returns a single value and can be used within SQL queries.
- Procedures improve code reuse and security by storing logic in the database. Functions enable computations and can be integrated in SQL.
- Both help in maintaining business logic centrally and reduce network traffic by running code on the database server.

STEP-BY-STEP ALGORITHM

- Start
- Connect to Oracle Database
- Create Stored Procedure IncreaseSalary with input parameters department_id and percentage
- Create Function GetAvgSalary with input parameter department_id that returns average salary
- Execute procedure to update salaries in a specific department
- Call function to get average salary of that department

- Display updated employee data and average salary
- End

QUESTIONS

- What is the difference between a stored procedure and a function in PL/SQL?
- How do you pass parameters to a PL/SQL procedure?
- Write a stored procedure to increase salary of employees by a fixed amount.
- Explain how a function can be used within an SQL query.
- How does exception handling work in PL/SQL?

CONCLUSION

Experiment No. 9

TITLE

Write and execute suitable database triggers considering row-level and statement-level triggers.

PROBLEM STATEMENT

Create and implement row-level and statement-level triggers in SQL to monitor and log actions performed on the employees table.

OBJECTIVE

- Understand the use and purpose of SQL triggers.
- Differentiate between row-level and statement-level triggers.
- Implement triggers to monitor INSERT, UPDATE, and DELETE operations.
- Log audit data into a separate table.
- Test trigger functionality using sample data.

PLATFORM REQUIRED

• Operating System: Windows or Linux
• Software/Tools: MySQL / Oracle SQL / PostgreSQL / SQL Developer

THEORY

A trigger is a stored procedure that automatically executes when an event (INSERT, UPDATE, DELETE) occurs on a table.

Types of Triggers:

• Row-level trigger: Executes once for each row affected.

• Statement-level trigger: Executes once for the entire SQL operation, regardless of row count.

STEP BY STEP ALGORITHM

- • Create base tables: employees and audit_log.
- • Insert sample data into employees.
- • Create a row-level trigger to log updates to salary.
- • Create a statement-level trigger to log general updates.
- • Perform UPDATE operations to test triggers.
- • Query the audit_log to verify trigger actions.

SQL CODE

Step 1: Create Tables

```
CREATE TABLE employees (
  emp_id INT PRIMARY KEY,
  name VARCHAR(50),
  salary DECIMAL(10,2),
  dept VARCHAR(30)
);
CREATE TABLE audit_log (
  log_id INT AUTO_INCREMENT PRIMARY KEY,
  emp_id INT,
  action VARCHAR(100),
  log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Step 2: Insert Data

```
INSERT INTO employees VALUES (1, 'Alice', 50000, 'HR');
INSERT INTO employees VALUES (2, 'Bob', 60000, 'IT');
```

Step 3: Row-Level Trigger

```
CREATE TRIGGER trg_row_salary_update
AFTER UPDATE ON employees
FOR EACH ROW
WHEN (OLD.salary <> NEW.salary)
BEGIN
  INSERT INTO audit_log(emp_id, action)
VALUES (OLD.emp_id, 'Salary updated');
END;
```

Step 4: Statement-Level Trigger

```
CREATE TRIGGER trg_statement_update
AFTER UPDATE ON employees
BEGIN
  INSERT INTO audit_log(emp_id, action)
VALUES (NULL, 'Employees table updated');
END;
```

Step 5: Perform Updates

```
UPDATE employees SET salary = salary + 1000 WHERE emp_id = 1;
UPDATE employees SET dept = 'Finance' WHERE emp_id = 2;
```

Step 6: View Audit Logs

```
SELECT * FROM audit_log;
```

QUESTIONS

- • What is the difference between row-level and statement-level triggers?
- • Can a trigger call another trigger?
- • What is the importance of the WHEN clause in a trigger?
- • Explain what happens when multiple rows are updated in a single statement.
- • When would you use a BEFORE vs AFTER trigger?
- • What are some practical use cases for triggers in real-world systems?

CONCLUSION

Experiment No. 10

TITLE

Create a MongoDB Document and Implement CRUD Operations on It.

PROBLEM STATEMENT

Design a MongoDB document schema and perform Create, Read, Update, and Delete (CRUD) operations on it using MongoDB Shell or Compass. The document will represent employee data and allow manipulation of the stored information through queries.

OBJECTIVE

- Understand document structure in MongoDB (NoSQL).
- Create a MongoDB database and collection.
- Insert one or more documents into the collection.
- Perform CRUD operations: Insert (Create), Find (Read), Update, Delete.
- Explore MongoDB Shell or Compass for interacting with the database.

PLATFORM REQUIRED

• Operating System: Windows or Linux
• Software/Tools: MongoDB, MongoDB Compass (GUI), MongoDB Shell or Node.js (optional)

THEORY

MongoDB is a NoSQL database that stores data in a document-oriented format called BSON (Binary JSON).

Each document is a key-value pair structure and is stored in collections.

CRUD refers to the four basic operations that can be performed on a document in the database:

Operation	Description
Create	Insert documents into a collection
Read	Retrieve documents from the collection
Update	Modify existing documents
Delete	Remove documents from the collection

STEP BY STEP ALGORITHM

- Start the MongoDB service.
- Connect to MongoDB using Mongo Shell or Compass.
- Create a new database 'company'.
- Create a collection 'employees'.
- Insert sample employee documents.
- Retrieve (read) all or specific documents using queries.
- Update selected fields in a document.
- Delete one or more documents.
- Optionally, use filtering, projections, and sorting.

MONGODB COMMANDS

Step 1: Start Mongo Shell
mongosh
Step 2: Create Database
use company
Step 3: Create & Insert Document into Collection

```
db.employees.insertOne({
  emp_id: 101,
  name: "Alice",
  department: "HR",
  salary: 50000
})

db.employees.insertMany([
  { emp_id: 102, name: "Bob", department: "IT", salary: 60000 },
  { emp_id: 103, name: "Charlie", department: "Finance", salary: 55000 }
])
```

Step 4: Read (Find) Data

```
db.employees.find()
db.employees.find({ department: "IT" })
db.employees.find({}, { name: 1, salary: 1 })
```

Step 5: Update Document

```
db.employees.updateOne(
  { emp_id: 103 },
  { $set: { salary: 58000 } }
)
db.employees.updateMany(
  { department: "IT" },
  { $inc: { salary: 2000 } }
)
```

Step 6: Delete Document

```
db.employees.deleteOne({ emp_id: 101 })
db.employees.deleteMany({ department: "HR" })
```

QUESTIONS

- What is the difference between SQL and NoSQL databases?
- Explain the document model of MongoDB.
- How does MongoDB store data internally?
- What is the difference between insertOne and insertMany?
- How do you perform conditional updates in MongoDB?
- Explain the usage of $set and $inc operators.
- What happens if you update a field that does not exist?

CONCLUSION